

RentIt

Del hvad du har. Lej hvad du behøver.

Platform til privat udlejning af genstande.

Processrapport

Titelblad

Projekt titel: RentIt

Projekt periode: 23.03.2026 - 29.04.2026

Skole: Technical Education Copenhagen (TEC) EUX

Uddannelse: Data og Kommunikation Uddannelse

Hovedvejleder: Flemming Sørensen

Hold: 1205hf26046xp

Deltagere: Chanakya Joshi, Subarna Gurung og Marko Dankovich

INDHOLD

Læsevejledning.....	1
Forord.....	1
Formål.....	1
Hvem er vi?.....	2
Chanakya Joshi.....	2
Subarna Gurung.....	2
Marko Dankovich.....	2
Problemformulering.....	2
Projektplanlægning.....	3
Projektstyring.....	3
Arbejdsfordeling.....	4
Estimeret tidsplan.....	4
Begrundelse for metodevalg og teknologi.....	5
ASP.NET Core Web API - Backend.....	5
Angular - Frontend.....	5
Entity Framework Core - ORM.....	5
SQL Server - Database.....	6
SignalR - Realtidskommunikation.....	6
JWT med Refresh Tokens - Autentificering.....	6
Docker Compose - Kørsel af projektet.....	6
Cloudinary - Billedlagring.....	6
Geoapify + OpenStreetMap.....	7
Repository Pattern - Arkitektur.....	7
Swagger - API dokumentation.....	7
GitHub - Versionsstyring.....	8
Beskrivelse af væsentlige elementer fra produktet.....	8
Rollesystem og JWT autentificering.....	8
Brugerscore og troværdighedssystem.....	9
Admin godkendelsesflow.....	10
Lånehåndtering og statusflow.....	11
Automatiske baggrundstjenester.....	12
Realtidskommunikation med SignalR.....	12
Tvister og konfliktløsning.....	13
Email bekræftelse og kontoaktivering.....	14
Verifikationssystem.....	14
Anmeldelsessystem (Reviews).....	14

Rapporteringssystem.....	15
Support Chat.....	15
Brugerblokeringsystem.....	15
Appeal-Systemet.....	15
Realiseret tidsplan.....	16
Konklusion.....	17
Gruppearbejde.....	17
Login og rollesystem.....	18
Admin godkendelse.....	18
Søgning og browse.....	18
Lejeproessen.....	19
Hvad vi kunne have gjort anderledes.....	19
Opsummering.....	19
Bilag 10 - Kildekode.....	20
Frontend: https://github.com/Chanaquid/Svendeproeven/tree/main/frontend.....	20
Forkortelse og begreber.....	21
Bilag oversigt.....	23

Læsevejledning

Denne rapport er processrapporten for RentIt og beskriver hvordan udviklingsforløbet har været, fra ide til færdigt produkt. Rapporten handler ikke om hvad systemet kan, men om hvordan vi arbejdede, hvilke teknologier vi valgte og hvorfor, og hvad vi lærte undervejs.

Rapporten er skrevet af Chanakya Joshi, Subarna Gurung og Marko Dankovich som en del af vores afsluttende projekt på TEC EUX Data og Kommunikation.

Rapporten er bygget op så den kan læses fra start til slut, men du kan også hoppe direkte til det afsnit du er interesseret i ved hjælp af indholdsfortegnelsen. Tekniske detaljer om selve produktet findes i produktrapporten. En samlet logbog fra alle tre gruppemedlemmer findes som bilag bagerst i denne rapport.

Kildekoden for både frontend og backend er tilgængelig på GitHub og kan findes som bilag.

Forord

Med denne rapport vil vi give et indblik i hvordan udviklingsforløbet for RentIt har været. Vi vil beskrive tankerne bag projektet, hvordan vi har samarbejdet som gruppe, hvilke teknologiske valg vi har truffet og begrundelserne herfor.

RentIt startede som en ide om at løse et problem vi alle kender til, man har ting stående derhjemme som man ikke bruger, og andre mangler præcis de samme ting. De eksisterende løsninger på markedet er enten dyre, begrænsede eller ustrukturerede. Det fik os til at tænke: hvad hvis man kunne bygge en platform hvor private kan leje og udleje ting direkte til hinanden på en nem og tryk måde?

Vi er tre elever på TEC EUX Data og Kommunikation der har arbejdet på dette projekt som vores afsluttende svendep prøve. Vi kommer alle med forskellig erfaring og styrker, og det har vi forsøgt at udnytte bedst muligt gennem hele projektforsløbet. I gennem projektet har vi lært meget, ikke kun teknisk, men også om hvad det kræver at samarbejde på tværs af frontend og backend, og hvad det vil sige at bygge et system der er større end hvad vi hver især har prøvet før.

Formål

Formålet med RentIt er at gøre det nemmere og trykgere for private at leje ting af hinanden. Platformen skal på den ene side give udlejere mulighed for at slå deres ting op og have kontrol over hvem der låner dem, og på den anden side give lejere en enkel måde at finde og booke det de mangler, uden at det bliver rodet eller usikkert.

For at understøtte dette har vi bygget et system med brugerverifikation, et scoringsbaseret troværdighedssystem, admin-godkendelse af opslag og lejeforespørgsler, samt et tviste- og rapporteringssystem der sikrer at platformen kan håndtere konflikter på en fair og struktureret måde.

Hvem er vi?

Vi er tre elever på TEC EUX Data og Kommunikation der har arbejdet sammen om dette projekt som vores afsluttende svendeprøve. Vi kommer alle med forskellige personligheder og interesser, og det har gjort gruppen stærkere.

Chanakya Joshi

Chanakya er en kreativ og analytisk person, der brænder for webudvikling, UI/UX design og data. Han tænker meget over hvordan tingene ser ud og føles fra brugerens perspektiv, og han er aldrig helt tilfreds før brugeroplevelsen sidder rigtigt. Udover design har han også en stor interesse for data og algoritmer, han kan godt lide at forstå hvordan ting fungerer under overfladen og ikke bare på overfladen.

Subarna Gurung

Subarna er struktureret, logisk og vedholdende. Han trives med at løse komplekse tekniske problemer og giver ikke op før tingene fungerer præcis som de skal. Han har et naturligt overblik over hvordan de forskellige dele af et system hænger sammen, og han er den i gruppen der tænker mest over systemarkitektur og hvordan man bygger noget der er robust og skalerbart.

Marko Dankovich

Marko er grundig, omhyggelig og detaljeorienteret. Han trives med at grave ned i detaljerne og finde ud af præcis hvorfor noget ikke virker som det skal. Han er den i gruppen der sørger for at tingene rent faktisk er testet ordentligt inden de erklæres færdige, og hans tålmodighed og systematiske tilgang har været uvurderlig for projektets kvalitet.

Problemformulering

Vores problem er, at der ikke findes en nem og tryk måde for private at leje ting af hinanden uden at det bliver rodet eller usikkert. Mange har ting liggende derhjemme som andre kunne bruge, men man tør ikke altid låne det ud, fordi man ikke har overblik, regler eller en måde at sikre sig på. Samtidig kan det også være svært for lejereren at finde ting hurtigt og holde styr på afleveringsdato.

Derfor har vi valgt at lave en web app, hvor brugere kan oprette en konto, lægge deres egne ting op til udlejning, browse og søge i ting som andre har lagt op, sende en leje-forespørgsel og få et overblik over hvad de lejer og hvornår de skal aflevere det.

For at undgå scam og dårligt indhold har vi også implementeret en admin rolle, som kan godkende eller afvise nye opslag før de bliver vist offentligt. Admin kan også se alle brugere og alle opslag og lejeaftaler, så man kan tjekke hvis der er problemer.

På baggrund af dette har vi arbejdet med følgende spørgsmål:

1. Hvordan kan vi lave et login og rollesystem (User/Admin), så kun rigtige brugere kan oprette opslag og sende leje-forespørgsler, og så admin har de rigtige rettigheder?
2. Hvordan kan vi lave admin-godkendelsen af nye opslag, så kun godkendte items bliver vist i browse sektionen, og brugeren altid ser de rigtige opslag?

3. Hvordan kan vi lave en søge og browse-funktion, så brugere nemt kan finde det de leder efter (fx kategori, navn, tilgængelighed), og samtidig holde systemet hurtigt og overskueligt?
4. Hvordan kan vi håndtere leje processen med forespørgsel og depositum, så det føles trykt for udlejeren, og så lejeren altid kan se status og afleveringsdato?

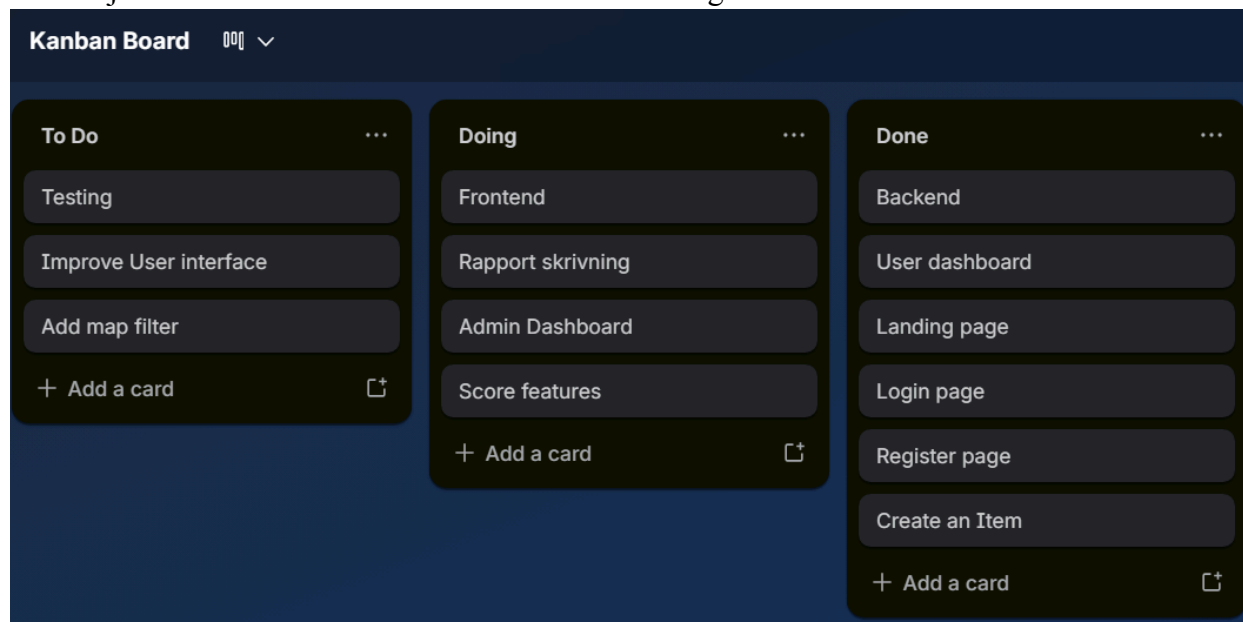
Projektplanlægning

Meget af projektplanlægningen skete allerede under udarbejdelsen af kravspecifikationen. Her blev use cases, funktionelle krav og systemets overordnede struktur defineret, hvilket gav os et godt udgangspunkt for at gå i gang med selve udviklingen.

Projektstyring

Vi har valgt at arbejde iterativt gennem hele projektforsløbet. Det har betydet at vi ikke har låst os fast på en stor plan fra starten, men i stedet har arbejdet i mindre bidder og løbende tilpasset os efterhånden som vi fik mere overblik over hvad der krævedes.

Til at holde styr på opgaverne har vi brugt Trello som kanban board. Her har vi oprettet opgaver for alle de ting der skulle laves, og flyttet dem fra "To Do" til "Doing" til "Done" efterhånden som arbejdet skred frem. Det har gjort det nemt for alle i gruppen at se hvad der arbejdes på lige nu, hvad der venter og hvad der er færdigt. Særligt de små ekstra opgaver der dukkede op undervejs er blevet skrevet ind i Trello så de ikke blev glemt.



Figur 1: Trello kanban board brugt til projektstyring gennem hele forløbet

Kommunikationen i gruppen er foregået primært via Discord, hvor vi har holdt løbende gruppemøder og opdateret hinanden om fremskridt og udfordringer. Det har fungeret godt at have et fast sted at mødes digitalt, da vi ikke altid har haft mulighed for at sidde fysisk sammen. Vi har desuden skrevet logbog dagligt, som dokumenterer hvad hver person har arbejdet på den pågældende dag. *Logbogen findes som bilag bagerst i rapporten.*

Arbejdsfordeling

Vi er tre i gruppen og har fordelt arbejdet ud fra vores individuelle styrker og interesser. Chanakya har primært haft ansvar for frontend, styling og brugeroplevelse i Angular samt projektdokumentation og rapportskrivning. Subarna har primært stået for backend, databasestruktur og API-logik i ASP.NET Core, og har desuden hjulpet med at forbinde backend og frontend så de to lag kommunikerer korrekt. Marko har arbejdet tæt med Subarna på backend, haft ansvar for at teste API endpoints i Swagger og Postman, og hjulpet med integrationen mellem frontend og backend.

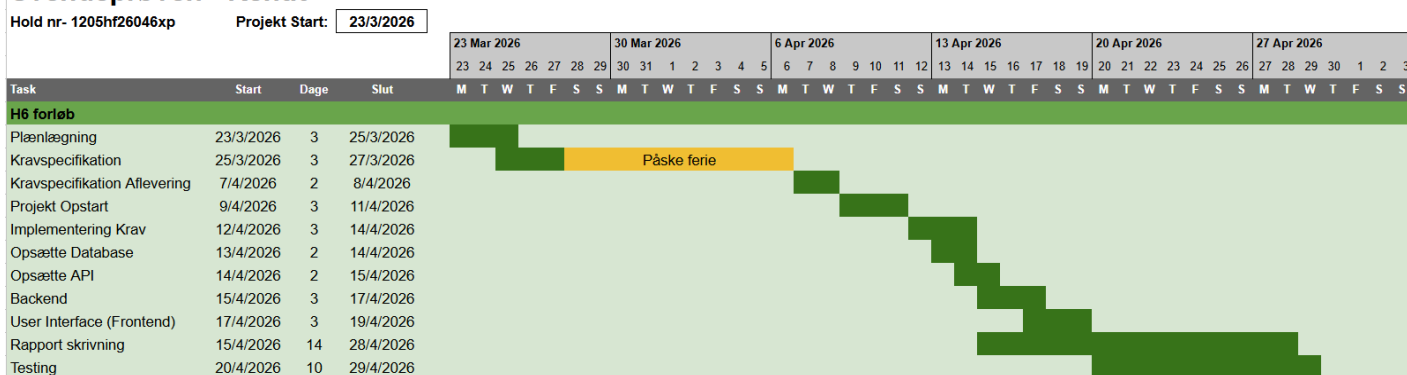
Selvom vi hver især har haft vores primære ansvarsområder, har vi hjulpet hinanden på tværs når der opstod behov for det. Det har været en stor fordel at arbejde i en gruppe frem for alene, da vi har kunnet vende problemer og ideer med hinanden og finde løsninger hurtigere end hvis vi hver havde siddet alene med det.

Estimeret tidsplan

Under udarbejdelsen af kravspecifikationen lavede vi en estimeret tidsplan for hele projektforsløbet.

Tidsplanen viser at arbejdet er struktureret så backend og database sættes op først, efterfulgt af frontend udvikling der bygger ovenpå de færdige API endpoints. Vi valgte bevidst denne rækkefølge fordi frontend er afhængig af at have stabile og testede API endpoints at kalde, det giver ikke mening at bygge brugergrænsefladen før man ved præcis hvilke data den skal arbejde med. Database og API opsætning er derfor placeret tidligt i forløbet som et fundament for resten. Rapport skrivning og testing løber parallelt med udviklingen i den sidste del af forløbet for at sikre at begge dele kan nå inden afleveringsfristen.

Svendeprøven - RentIt



Figur 2: GANTT diagram Estimeret tidsplan (Bilag 5)

Begrundelse for metodevalg og teknologi

I dette afsnit beskriver vi hvilke teknologier vi har valgt at bruge i RentIt og hvorfor vi valgte netop dem. Vores valg er primært baseret på hvad vi havde erfaring med i forvejen fra vores uddannelse, men vi har også taget hensyn til hvad der giver god mening teknisk set for denne type projekt.

ASP.NET Core Web API - Backend

Til backend har vi valgt ASP.NET Core Web API med C#. Det er det backend framework, vi har mest erfaring med fra vores uddannelse, og det er bygget til præcis den type REST API vi har brug for. ASP.NET Core oversætter automatisk API svar til JSON objekter uden ekstra opsætning, og det fungerer på tværs af platforme - Windows, macOS, Linux og Docker ¹.

Vi valgte C# og ASP.NET Core frem for alternativer som Node.js eller Django fordi vi kendte det bedst og vidste at vi kunne bygge det system vi havde i tankerne inden for den tid vi havde til rådighed. At vælge kendte teknologier har betydet, at vi har kunnet fokusere på at løse de tekniske udfordringer i selve projektet frem for at lære et nyt framework fra bunden.

Angular - Frontend

Til frontend har vi valgt Angular som framework. Angular er et komponentbaseret framework der gør det nemt at bygge Single Page Applications ², hvilket passer perfekt til RentIt da vi ønsker en hurtig og flydende brugeroplevelse uden at siden genindlæser sig selv ved hver navigation.

Vi valgte Angular frem for andre alternativer som React eller Vue fordi vi havde mest erfaring med Angular fra vores uddannelse. Angular er desuden et meget struktureret framework med en klar måde at organisere koden på med komponenter, services og dependency injection, hvilket gør koden nem at vedligeholde og udvide. TypeScript som Angular bruger som standard giver også bedre fejlhåndtering og gør koden mere overskuelig.

Entity Framework Core - ORM

Vi bruger Entity Framework Core som ORM til at kommunikere med databasen. EF Core gør det muligt at arbejde med databasen gennem C# objekter i stedet for at skrive rå SQL ³, hvilket reducerer mængden af kode og gør det nemmere at vedligeholde. Vi har valgt Code-First tilgangen, hvilket betyder at vi definerer vores modeller i C# og lader EF Core generere databasetabellerne automatisk via migreringer. Det har gjort det nemt at ændre og tilføje til databasestrukturen undervejs i udviklingen.

¹ <https://learn.microsoft.com/en-us/aspnet/core/introduction-to-aspnet-core>

² <https://angular.dev/overview>

³ <https://learn.microsoft.com/en-us/ef/core/>

SQL Server - Database

Til database har vi valgt SQL Server. Det er en relationel database der passer godt til vores system, da vi har mange relationer mellem brugere, opslag, lejeaftaler og andre entiteter. SQL Server integrerer problemfrit med Entity Framework Core og ASP.NET Core, og vi havde erfaring med det fra vores uddannelse.

SignalR - Realtidskommunikation

For at implementere realtidskommunikation i chat og notifikationer har vi valgt SignalR. SignalR er et bibliotek der er bygget ind i ASP.NET Core og gør det nemt at sende beskeder fra server til klient i realtid ⁴ uden at siden skal genindlæses. Det har vi brugt til både lånechat, direkte beskeder og notifikationer, så brugerne modtager opdateringer øjeblikkeligt uden at skulle opdatere siden manuelt.

Vi valgte SignalR frem for andre realtidsløsninger fordi det er integreret direkte i ASP.NET Core og derfor var nemt at sætte op uden at skulle introducere et helt nyt eksternt system.

JWT med Refresh Tokens - Autentificering

Til autentificering bruger vi JWT tokens kombineret med refresh tokens. JWT tokens sikrer at kun loggede ind brugere kan tilgå beskyttede endpoints, og refresh tokens sørger for at brugeren ikke bliver logget ud unødvendigt. Vi valgte JWT fordi det er en industristandard for token-baseret autentificering i REST APIs ⁵ og fordi det integrerer naturligt med ASP.NET Core Identity.

Docker Compose - Kørsel af projektet

Vi har valgt at bruge Docker Compose til at køre hele projektet samlet. Med en enkelt kommando starter Docker Compose frontend, backend og database op i separate containers ⁶. Det har gjort det meget nemmere at samarbejde som gruppe, da alle tre gruppemedlemmer kan køre præcis det samme miljø på deres egen computer uanset styresystem. Det eliminerer problemet med at noget virker på en persons computer men ikke på en andens.

Cloudinary - Billedlagring

Til lagring og håndtering af billeder har vi valgt Cloudinary. Cloudinary er en cloud-baseret medietjeneste der gør det muligt at uploade, transformere og levere billeder via et CDN ⁷ uden at vi selv skal håndtere fillagring på serveren.

Vi bruger Cloudinary til tre forskellige formål i RentIt: billeder på item opslag, bevisbilleder i tvistesager og dokumenter uploadet i forbindelse med brugerverifikation. At have al billedhåndtering samlet et sted har gjort det nemt at arbejde med på tværs af hele systemet.

⁴ <https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction?view=aspnetcore-10.0>

⁵ <https://jwt.io/introduction>

⁶ <https://docs.docker.com/compose/>

⁷ <https://cloudinary.com/documentation>

Vi valgte Cloudinary frem for at gemme billeder lokalt på serveren fordi lokal fillagring ikke skalerer godt og skaber problemer i et Docker-miljø hvor containere kan genskabes og lokale filer dermed mistes. Cloudinary løser dette ved at billeder ligger eksternt og altid er tilgængelige via en URL uanset serverens tilstand. Derudover tilbyder Cloudinary en gratis tier der dækker vores behov i dette projekt, og integrationen med ASP.NET Core er ligetil via deres officielle SDK.

Geoapify + OpenStreetMap

Til kortvisning og afstandsfiltrering i browse sektionen har vi valgt Geoapify og OpenStreetMap. OpenStreetMap leverer det åbne kortmateriale som vises i brugergrænsefladen, mens Geoapify bruges til geokodning, det vil sige at konvertere adresser til koordinater så opslag kan placeres geografisk på kortet ⁸. Vi valgte disse løsninger frem for Google Maps fordi begge er gratis at bruge inden for vores behov og ikke kræver et betalt API-abonnement. Da disse tjenester er eksterne og ikke kræver vores JWT token, er de bevidst ekskluderet fra Angular HTTP interceptoren ⁹.

Repository Pattern - Arkitektur

I backend har vi valgt at strukturere koden efter repository pattern med en tydelig lagdeling: Controllers → Services → Repositories → DbContext. Det betyder at hver del af systemet har et klart ansvar, controllerne håndterer HTTP-kald og returnerer svar, services indeholder forretningslogikken, og repositories er det eneste sted der kommunikerer direkte med databasen via Entity Framework Core.

Vi valgte denne arkitektur frem for at skrive al logik direkte i controllerne fordi det gør koden markant nemmere at vedligeholde og udvide. Når et nyt krav dukker op, ved vi præcis hvilken del af koden der skal ændres. Det har også betydet at backend og frontend har kunnet udvikles mere uafhængigt af hinanden, da services eksponerer klare interfaces som controllerne arbejder imod.

Alle services er defineret gennem interfaces, hvilket er i tråd med SOLID-principperne og gør det nemt at udskifte implementationer hvis behovet opstår.

Swagger - API dokumentation

Til dokumentation og test af vores API endpoints har vi brugt Swagger. Swagger genererer automatisk en interaktiv dokumentationsside ud fra vores API kode, hvor man kan se alle endpoints og teste dem direkte i browseren. Det har været særligt nyttigt for samarbejdet mellem frontend og backend, da Marko og Subarna nemt kunne teste endpoints mens Chanakya kunne se præcis hvilke data han skulle forvente fra API'et.

⁸ <https://apidocs.geoapify.com/>

⁹ <https://nominatim.openstreetmap.org/ui/about.html>

GitHub - Versionsstyring

Vi har valgt at bruge GitHub til versionsstyring og samarbejde. GitHub giver os mulighed for at arbejde på koden samtidigt uden at vi forstyrrer hinandens arbejde. Vi har arbejdet med branches, så hver person har kunnet arbejde på sin del af projektet uden at påvirke hovedkoden. Når en feature var klar, mergede vi den ind i main-branchen.

GitHub har været særligt vigtigt for os som gruppe på tre, fordi risikoen for konflikter i koden er større når flere arbejder på det samme projekt. Med GitHub har vi altid haft et struktureret overblik over hvad der er ændret, hvornår og af hvem. Det har også fungeret som en sikkerhed, hvis noget gik galt kunne vi altid gå tilbage til en tidligere version af koden.

Beskrivelse af væsentlige elementer fra produktet

I dette afsnit beskriver vi hvordan vi har løst nogle af de vigtigste og mest udfordrende tekniske elementer i RentIt. Det er ikke en komplet gennemgang af alle funktioner, men en beskrivelse af de dele der har krævet mest tanke og arbejde at få til at fungere.

Rollesystem og JWT autentificering

Et af de første og vigtigste elementer vi skulle have på plads var login og rollesystemet. Vi bruger ASP.NET Identity til at håndtere brugere og passwords, og JWT tokens til at autentificere brugere når de kalder vores API.

Når en bruger logger ind udsteder backend et JWT token og et refresh token. JWT tokenet sendes med ved alle efterfølgende API kald i headeren, og backend validerer tokenet før det overhovedet når frem til controllerne. Hvis tokenet er ugyldigt eller mangler returneres en 401 Unauthorized fejl. Hvis brugeren er logget ind men ikke har den rette rolle, fx en almindelig bruger der forsøger at tilgå et admin endpoint, returneres en 403 Forbidden fejl.

I Angular bruger vi en HTTP interceptor der automatisk tilføjer JWT tokenet til alle API kald, så vi ikke skal gøre det manuelt for hvert enkelt kald. Refresh token logikken sørger for at brugeren automatisk får et nyt token når det gamle udløber, så man ikke bliver logget ud midt i en session.

```

src > app > services > TS authInterceptorService.ts > ...
1  import { HttpInterceptorFn } from '@angular/common/http';
2  import { inject } from '@angular/core';
3  import { AuthService } from './authService';
4
5  export const authInterceptorService: HttpInterceptorFn = (req, next) => {
6
7      if (req.url.startsWith('https://api.geoapify.com') ||
8          req.url.startsWith('https://nominatim.openstreetmap.org')) {
9          return next(req);
10     }
11
12     const token = inject(AuthService).getToken();
13     if (token) {
14         req = req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });
15     }
16     return next(req);
17 };

```

Figur 3:JWT Interceptor

HTTP interceptoren håndterer automatisk at JWT tokenet tilføjes til alle API kald uden at vi skal gøre det manuelt i hver enkelt komponent. Vi har bevidst valgt at ekskludere eksterne tredjeparts services som Geoapify og OpenStreetMap fra interceptoren, da disse ikke kræver vores JWT token. Det betyder at al kommunikation med vores eget backend er sikret, mens kortdata og geolokation stadig kan hentes frit.

Brugerscore og troværdighedssystem

Et af de mere unikke elementer i RentIt er brugerscore systemet. Alle brugere starter med 100 point, og scoren påvirkes løbende af brugerens adfærd på platformen. Man får point for at aflevere til tiden og mister point for forsinkede afleveringer, skader og aflysninger.

Scoren er ikke bare et tal, den har direkte konsekvenser for hvad brugeren kan gøre i systemet. Brugere med en score over 50 kan sende lejeforespørgsler direkte til udlejeren. Brugere med en score mellem 20 og 50 skal have deres forespørgsler godkendt af admin først. Brugere der falder under 20 point bliver blokeret fra at låne overhovedet.

Al scorehistorik gemmes i databasen så brugeren og admin altid kan se hvad der har påvirket scoren og hvornår. Dette giver gennemsigtighed og gør det muligt at anke afgørelser man er uenig i.

```

public enum BorrowingStatus
{
    Free = 0,           // Score over 50 – kan låne frit
    AdminApproval = 1, // Score 20–50 – kræver admin godkendelse
    Blocked = 2         // Score under 20 – kan ikke låne
}

//Loan Status
public enum LoanStatus
{
    Pending = 0,           // Afventer udlejerens svar
    AdminPending = 1,      // Afventer admin godkendelse (lav score)
    Approved = 2,          // Godkendt – låntager kan hente genstanden
    Active = 3,            // Genstanden er afhentet
    Completed = 4,         // Genstanden er returneret
    Late = 5,              // Ikke returneret til tiden
    Rejected = 6,          // Afvist af admin eller udlejer
    Cancelled = 7,         // Annulleret af låntager
    Extended = 8           // Forlænget efter aftale
}

```

Figur 4: BorrowingStatus + LoanStatus

De to enums illustrerer hvordan brugerscore og lånestatus er direkte forbundet i systemet. BorrowingStatus bestemmer hvilken vej en lejeforespørgsel tager, høj score går direkte til udlejeren, lav score går til admin godkendelse, og meget lav score blokerer brugeren helt. LoanStatus viser det fulde livsforløb for et lån fra forespørgsel til aflevering, og hver statusændring udløser automatisk en notifikation til begge parter via SignalR.

Admin godkendelsesflow

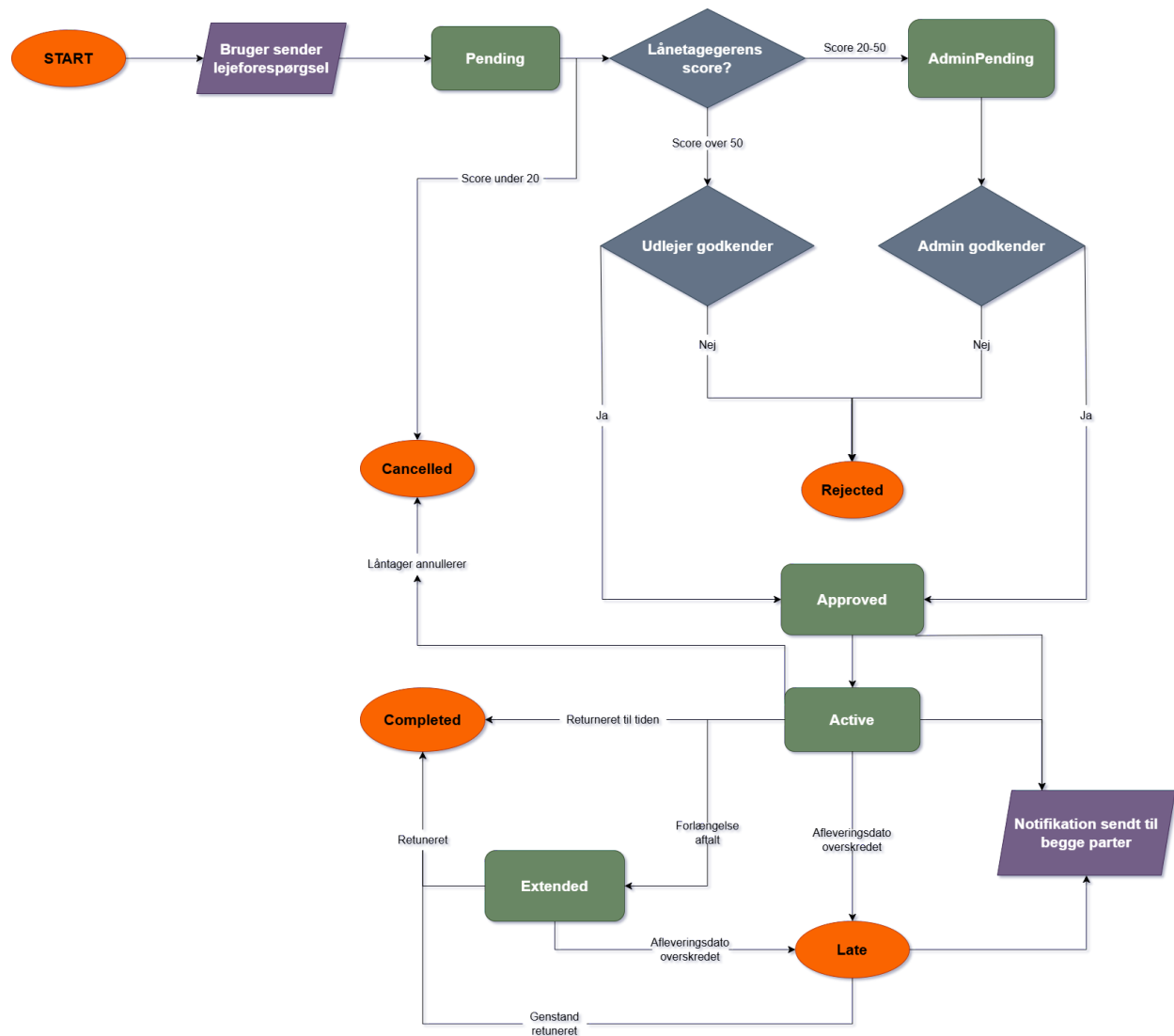
Et centralt element i systemet er admin godkendelsen af nye opslag. Når en bruger opretter et opslag får det automatisk status "Pending" og bliver ikke vist i browse sektionen. Admin kan herefter gennemgå opslaget og enten godkende eller afvise det med en kommentar.

Kun opslag med status "Approved" vises i browse sektionen. Det sikrer at platformen ikke bliver fyldt med falske eller upassende opslag. Hvis et opslag afvises kan brugeren rette det og sende det til godkendelse igen.

Det samme godkendelsesflow gælder for lejeforespørgsler fra brugere med lav score, disse skal igennem admin godkendelse inden udlejeren overhovedet ser dem.

Lånehåndtering og statusflow

Lånehåndteringen er et af de mest komplekse elementer i systemet fordi en leje gennemgår mange forskellige statusser fra start til slut. Et lån starter som "Pending" når forespørgslen sendes, og kan herefter gå igennem statusser som "AdminPending", "Approved", "Active", "Completed", "Late", "Rejected" og "Cancelled".



Figur 5: LoanStatus flow diagram

Figur 5 illustrerer det fulde statusflow for et lån. Et lån starter altid som Pending når lejeforespørgslen sendes. Afhængigt af låntagerens brugerscore tager forespørgslen forskellige veje, brugere med høj score går direkte til udlejeren, brugere med middel score skal igennem admin godkendelse, og brugere med meget lav score blokeres helt og forespørgslen annulleres automatisk. Når et lån er godkendt og aktivt håndterer baggrundstjenesten automatisk overgangen til Late hvis afleveringsdatoen overskrides, og begge parter modtager en notifikation ved enhver statusændring.

Hver statusændring udløser en notifikation til de relevante brugere via SignalR, så begge parter altid er opdaterede om hvad der sker med lejeaftalen. Når et lån er aktivt kan låntager anmode om en forlængelse af lejeperioden, som udlejeren kan acceptere eller afvise.

Automatiske baggrundstjenester

For at systemet kan køre uden at nogen skal holde øje med det manuelt, har vi implementeret en række automatiske baggrundstjenester. Disse kører i baggrunden på serveren og udfører opgaver med faste intervaller.

Den vigtigste er den service der automatisk markerer lån som forsinkede når afleveringsdatoen er overskredet. Når et lån markeres som forsinket trækkes der automatisk point fra låntagerens score, og begge parter modtager en notifikation. Vi har også automatiske services der lukker inaktive support tråde, udløber afventende lejeforespørgsler og ophæver midlertidige brugerbans når banlængden er udløbet.

```
// Active loans past their EndDate with no return
var overdueLoans = await _loanRepository.GetOverdueActiveLoansAsync();

foreach (var loan in overdueLoans)
{
    loan.Status = LoanStatus.Late;
    _loanRepository.Update(loan);

    // -5 per day late, max -15 per loan, floor 0 - first day penalty
    var daysLate = (int)(DateTime.UtcNow.Date - loan.EndDate.Date).TotalDays;
    var pointsToDeduct = Math.Min(daysLate * 5, 15);
    var newScore = Math.Max(loan.Borrower.Score - pointsToDeduct, 0);

    await notificationService.SendAsync(
        loan.BorrowerId,
        NotificationType.LoanOverdue,
        $"Your loan for '{loan.Item?.Title}' is overdue. Please return it as soon as possible.",
        loan.Id,
        NotificationReferenceType.Loan
    );
}
```

Figur 6: AutoMarkLoansLateService

Dette uddrag viser kernelogikken i den vigtigste baggrundstjeneste. Hvert 30. minut hentes alle aktive lån der har overskredet afleveringsdatoen, status opdateres automatisk til Late, point trækkes fra låntagerens score baseret på antal forsinkede dage med et loft på 15 point, og begge parter notificeres øjeblikkeligt, alt uden at admin skal gøre noget manuelt.

Realtidskommunikation med SignalR

Chat og notifikationer i RentIt er bygget med SignalR som giver realtidskommunikation mellem server og klient. Det betyder at når en bruger modtager en besked eller en notifikation, vises den øjeblikkeligt uden at siden skal genindlæses.

Vi har fire typer kommunikation via SignalR, lånechat der er knyttet til en specifik lejeaftale, direkte beskeder mellem brugere, systemnotifikationer der sendes ved alle vigtige hændelser som godkendelser, statusændringer og nye beskeder, samt support chat hvor brugere kan kontakte admin direkte i realtid.

I Angular bruger vi en SignalR service der opretter forbindelsen til serveren og lytter efter indkommende beskeder. Når en besked modtages opdateres brugergrænsefladen automatisk.

```
public async Task JoinLoan(int loanId)
{
    var userId = Context.UserIdentifier;
    if (userId == null) return;

    var isAllowed = await _messageService.IsPartyToLoanAsync(loanId, userId);
    if (!isAllowed)
    {
        await Clients.Caller.SendAsync("Error", "Not allowed");
        return;
    }

    await Groups.AddToGroupAsync(Context.ConnectionId, $"loan_{loanId}");
    _onlineTracker.AddToLoan(Context.ConnectionId, userId, loanId);

    var unread = await _loanMessageRepository.GetUnreadMessagesForUserAsync(loanId, userId);
    if (unread.Any())
    {
        foreach (var msg in unread)
            msg.IsRead = true;

        await _loanMessageRepository.SaveChangesAsync();

        foreach (var msg in unread)
        {
            await Clients
                .Group($"loan_{loanId}")
                .SendAsync("MessageRead", msg.Id);
        }
    }
}
```

Figur 7: LoanChatHub

Når en bruger tilslutter sig en lånechat tjekker hubben først om brugeren faktisk er part i det pågældende lån, er de ikke det, afvises forbindelsen øjeblikkeligt. Herefter tilmeldes brugeren en SignalR gruppe specifik for det lån, og alle ulæste beskeder markeres automatisk som læste så begge parter altid ser korrekt læsestatus uden at skulle genindlæse siden.

Twister og konfliktløsning

Twistesystemet er et element vi ikke havde planlagt i den originale kravspec men som vi valgte at implementere fordi det er en naturlig del af en udlejningsplatform. Hvis en genstand kommer tilbage beskadiget kan enten udlejer eller låntager åbne en tvist og uploade bevisbilleder. Den

anden part har 72 timer til at svare med sin version af sagen. Herefter gennemgår admin begge sider og afgiver en dom der kan resultere i bøder eller scoreændringer.

Tvistesystemet giver platformen et ekstra lag af troværdighed og sikkerhed, fordi brugerne ved at der er en fair proces hvis noget går galt.

Email bekræftelse og kontoaktivering

Som en del af registreringsflowet kræver RentIt at nye brugere bekræfter deres emailadresse inden de kan logge ind og bruge platformen. Når en bruger opretter en konto sendes der automatisk en bekræftelsesmail med et unikt link. Brugeren aktiverer herefter kontoen ved at klikke på linket, og først derefter er login muligt.

Dette er implementeret via vores **IEmailService** og en dedikeret **ConfirmEmail** komponent i Angular. Vi valgte at kræve emailbekræftelse for at sikre at brugerne på platformen er reelle og at de email-adresser der er knyttet til konti faktisk tilhører brugerne. Det giver et ekstra lag af troværdighed og reducerer risikoen for oprettelse af falske konti.

Verifikationssystem

For at give brugerne endnu større tillid til hinanden har vi implementeret et frivilligt verifikationssystem. Brugere kan uploade et identitetsdokument- pas, nationalt ID-kort eller kørekort, som admin herefter gennemgår og enten godkender eller afviser med en kommentar.

Når en bruger bliver verificeret modtager de et verificeret-mærke på deres profil samt en score bonus. Verificerede brugere signalerer over for udlejere at de er reelle personer med en bekræftet identitet, hvilket øger sandsynligheden for at lejeforespørgsler bliver accepteret.

Dokumenterne uploades via Cloudinary og er kun tilgængelige for admin. Hele flowet fra brugerens ansøgning til adminens afgørelse understøttes af notifikationer via SignalR, så brugeren altid er opdateret om status på deres ansøgning.

Anmeldelsessystem (Reviews)

Efter en lejeaftale er afsluttet har begge parter mulighed for at anmelde hinanden og selve genstanden. Vi har implementeret to separate anmeldelsessystemer- UserReview og ItemReview, der begge er knyttet til et specifikt afsluttet lån.

Brugeranmeldelser giver lejere og udlejere mulighed for at vurdere hinandens adfærd og pålidelighed, hvilket over tid opbygger en synlig historik på brugerens profil. Itemanmeldelser giver lejere mulighed for at vurdere om genstanden levede op til beskrivelsen. Begge typer anmeldelser er kun mulige én gang per lån for at undgå misbrug.

Rapporteringssystem

For at holde platformen sikker og fri for misbrug har vi implementeret et rapporteringssystem hvor brugere kan indberette problematisk indhold eller adfærd. Det er muligt at rapportere andre brugere, opslag, anmeldelser og beskeder, og der kan angives en specifik årsag som falsk identitet, svindel, chikane, falsk opslag, vildledende beskrivelse eller spam.

Rapporter havner i en dedikeret admin-kø med statusser der går fra Pending over UnderReview til Resolved eller Dismissed. Admin kan gennemgå rapporten og tage de nødvendige handlinger, hvorefter reporteren modtager en notifikation om udfaldet.

Systemet er med til at give brugerne en måde at reagere på dårlig oplevelser udover tvistesystemet, og det giver admin et struktureret overblik over potentielle problemer på platformen.

Support Chat

Udover lånechat og direkte beskeder har vi implementeret et separat support chat system hvor brugere kan kontakte admin direkte ved hjælp af en dedikeret SupportChatHub via SignalR.

Supporttråde har tre statusser- Open, Claimed og Closed. Når en bruger opretter en supporttråd er den synlig for alle admins i en fælles kø. En admin kan herefter claime tråden og overtage ejerskabet af samtalen, så flere admins ikke svarer på samme henvendelse. Når sagen er løst kan admin lukke tråden.

Vi har desuden implementeret en automatisk baggrundstjeneste der lukker supporttråde der har været inaktive i en given periode, så køen ikke fyldes op med forladte henvendelser.

Brugerblokeringsystem

Brugere på RentIt har mulighed for at blokere andre brugere hvis de ikke ønsker kontakt med dem. En blokeret bruger kan hverken sende direkte beskeder eller lejeforespørgsler til den bruger der har blokeret dem. I Angular har vi en dedikeret side der viser brugerens blokerede brugere, og hvorfra blokeringen kan ophæves.

Systemet giver brugerne kontrol over deres egne interaktioner på platformen og er et supplement til rapporteringssystemet i tilfælde hvor en bruger blot ønsker at undgå en bestemt person uden at eskalere til admin.

Appeal-Systemet

Brugere der er uenige i en scoreændring eller en udstedt bøde har mulighed for at indgive en klage via appeal-systemet. Der findes to typer appeals- score appeals og fine appeals- og begge havner i en admin-kø hvor de gennemgås individuelt.

Når admin behandler en appeal kan de godkende eller afvise den. Ved godkendelse af en score appeal justeres brugerens score tilsvarende. Ved godkendelse af en fine appeal kan admin enten annullere bøden helt eller fastsætte et nyt beløb. Begge udfald udløser en notifikation til brugeren.

For at forhindre misbrug af systemet har vi implementeret en cooldown mekanisme- hvis en score appeal bliver afvist, kan brugeren ikke indgive en ny score appeal i en given periode. Dette er registreret på brugeren via feltet LastScoreAppealRejectedAt.

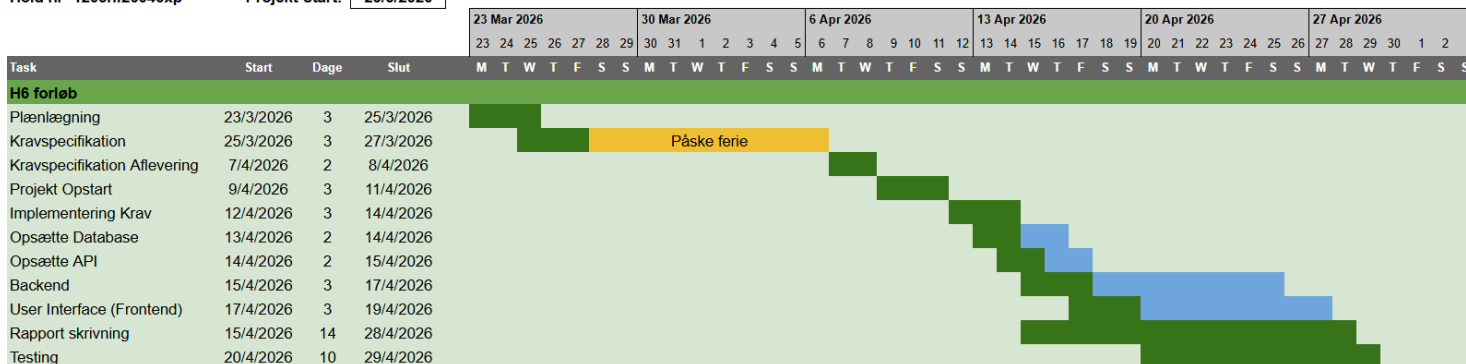
Realiseret tidsplan

Herunder ses den realiserede tidsplan sammenlignet med den oprindelige estimerede tidsplan. Overordnet set har projektet fulgt planen rimeligt godt, men der er enkelte afvigelser som vi vil forklare nedenfor.

Svendeprøven - RentIt

Hold nr- 1205hf26046xp

Projekt Start: 23/3/2026



Figur 8: Estimeret tidsplan og realiseret tidsplan

Afvielser og forklaring

Overordnet set har vi fulgt den estimerede tidsplan godt i starten af forløbet. Planlægning, kravspecifikation, projekt opstart og implementering af krav forløb alle som planlagt. Afvigelserne opstod primært i backend og frontend faserne.

Backend tog længere tid end estimeret. Den største årsag hertil var udfordringer med at merge kode på tværs af gruppemedlemmerne. Da vi arbejdede parallelt på backend og løbende tilføjede nye features, opstod der konflikter i GitHub som tog tid at løse korrekt uden at ødelægge eksisterende funktionalitet. Særligt i de dele af systemet hvor backend og frontend skulle forbindes krævede det ekstra koordination og fejlfinding.

User Interface forskubbete sig tilsvarende som følge af forsinkelsen i backend. Vi valgte bevidst at vente på at backend endpoints var fuldt testet og klar inden vi forbandt dem til frontend, hvilket betød at noget af frontend arbejdet måtte vente. Det var den rigtige beslutning for kvalitetens skyld, men det påvirkede tidsplanen.

Rapport skrivning og testing forløber som planlagt og vi forventer at nå afleveringsfristen den 29. april 2026.

Hvis vi skulle starte forfra ville vi fra dag et have defineret en klarere branch-strategi, hvor frontend og backend arbejdede på separate branches med aftalte merge-vinduer, så konflikter blev løst løbende i stedet for at hobe sig op.

Konklusion

På baggrund af vores gruppearbejde, projektets udformning samt den opstillede problemformulering kan vi konkludere at vores målsætning om at besvare følgende spørgsmål er opfyldt:

1. Hvordan kan vi lave et login og rollesystem (User/Admin), så kun rigtige brugere kan oprette opslag og sende leje-forespørgsler, og så admin har de rigtige rettigheder?
2. Hvordan kan vi lave admin-godkendelsen af nye opslag, så kun godkendte items bliver vist i browse sektionen, og brugeren altid ser de rigtige opslag?
3. Hvordan kan vi lave en søge og browse-funktion, så brugere nemt kan finde det de leder efter (fx kategori, navn, tilgængelighed), og samtidig holde systemet hurtigt og overskueligt?
4. Hvordan kan vi håndtere leje processen med forespørgsel og depositum, så det føles trygt for udlejer, og så lejeren altid kan se status og afleveringsdato?

Begrundelsen for denne konklusion er uddybet nedenfor.

Gruppearbejde

Vi har haft et rigtig godt samarbejde gennem hele projektforsløbet, og vi mener at netop samarbejdet har været en af de vigtigste faktorer for at projektet er gået så godt som det har. Fra første dag har vi haft en klar fordeling af ansvar, men vi har aldrig lukket af for hinanden, tværtimod har vi hele tiden hjulpet på tværs af frontend og backend når der opstod behov for det.

Den daglige kommunikation via Discord har betydet at ingen i gruppen nogensinde stod alene med et problem i lang tid. Når Marko fandt fejl i API endpoints under testning, blev de hurtigt rapporteret til Subarna og rettet samme dag. Når Chanakya manglede data fra backend til at færdiggøre en komponent i Angular, kunne han hurtigt få afklaret hvilke endpoints der var klar og hvilke der manglede. Den slags løbende kommunikation har gjort at vi har undgået de lange ventetider der ellers nemt kan opstå når frontend og backend udvikles sideløbende.

Kanban boardet i Trello har været et effektivt værktøj til at holde overblik over hvad der skulle laves, hvad der var i gang og hvad der var færdigt. Særligt de mange ekstra features der dukkede op undervejs, som verifikationssystemet, rapporteringssystemet, support chat og baggrundstjenesterne, er blevet skrevet ind i boardet med det samme så de ikke blev glemt. Det har givet os en god struktur og sikret at vi ikke mistede overblikket selvom projektet voksede sig større end planlagt.

Logbøgerne har også været nyttige, ikke kun som dokumentation, men som en måde at reflektere over hvad vi lavede hver dag og sikre at vi holdt fremdriften. At skrive hvad man har lavet tvinger en til at tænke over om man faktisk har fået nok ud af dagen, og det har hjulpet os til at holde fokus.

Login og rollesystem

Et af de første og vigtigste elementer vi skulle have på plads var login og rollesystemet, da næsten alt andet i systemet afhænger af at vi ved hvem brugeren er og hvilke rettigheder de har. Vi har implementeret et fuldt fungerende login og rollesystem med ASP.NET Identity og JWT tokens med refresh token support, samt emailbekræftelse ved oprettelse af ny konto.

ASP.NET Identity håndterer automatisk password hashing så adgangskoder aldrig gemmes i klar tekst i databasen. JWT tokens sikrer at kun loggede ind brugere kan tilgå beskyttede endpoints, og refresh tokens sørger for at brugeren ikke bliver logget ud midt i en session. I Angular bruger vi en HTTP interceptor der automatisk tilføjer JWT tokenet til alle API kald, samt route guards der forhindrer uautoriserede brugere i at tilgå beskyttede sider direkte via URL.

Rollesystemet med User og Admin fungerer præcis som planlagt. Admin endpoints er beskyttet så en almindelig bruger får 403 Forbidden hvis de forsøger at tilgå dem, både i backend og i frontend hvor admin sider er skjult bag en dedikeret AdminGuardService. Vi har testet dette grundigt og vurderer at rollesystemet er implementeret solidt og sikkert.

Admin godkendelse

Admin godkendelsesflowet er et af de elementer vi er mest tilfredse med i systemet. Vi har implementeret et flow hvor nye opslag automatisk får status "Pending" når de oprettes, og de vises ikke i browse sektionen før en administrator har gennemgået dem og godkendt dem.

Admin har et dedikeret panel med en godkendelseskø hvor alle ventende opslag kan ses med alle detaljer- titel, beskrivelse, kategori, pris, depositum og billeder. Admin kan godkende eller afvise et opslag, og ved afvisning kan der gives en kommentar til brugeren om hvorfor. Brugeren modtager en notifikation om afgørelsen og kan rette opslaget og sende det til godkendelse igen.

Det samme princip gælder for lejeforespørgsler fra brugere med lav score, som skal igennem admin godkendelse inden udlejer overhovedet ser dem. Vi vurderer at admin godkendelsessystemet er et af de stærkeste og mest gennemtænkte elementer i RentIt.

Søgning og browse

Browse sektionen viser kun godkendte opslag og understøtter filtrering på kategori, pris, stand og tilgængelighed samt søgning på titel. Systemet er bygget med paginering så det forbliver hurtigt og responsivt selv når der er mange opslag i databasen.

Vi har også implementeret afstandsfiltrering baseret på brugerens lokation, en "senest set" funktion der automatisk holder styr på hvilke opslag brugeren har kigget på, samt en favoritfunktion hvor brugeren kan gemme opslag de er interesserede i. Derudover har vi en kortvisning hvor brugere kan se opslag placeret geografisk. Vi er tilfredse med hvordan browse og søgning er blevet løst og mener at det lever op til kravet om at gøre det nemt og hurtigt at finde det man leder efter.

Lejeprocesen

Lejeprocesen er kernen i RentIt og det element der har krævet mest arbejde at få til at fungere godt. Vi har implementeret et komplet lejeflow hvor brugere kan sende lejeforespørgsler med start og slutdato, og udlejer kan acceptere eller afvise dem. Systemet registrerer depositum og viser altid aktuel status og afleveringsdato til begge parter. Låntagere kan desuden anmode om forlængelse af lejeperioden, som udlejer kan acceptere eller afvise.

Brugerscore systemet spiller en direkte rolle i lejeprocesen, brugere med høj score kan sende forespørgsler direkte til udlejer, mens brugere med lav score skal igennem admin godkendelse først, og brugere der falder under en kritisk grænse blokeres fra at låne overhovedet. Det skaber et reelt incitament til at opføre sig ordentligt på platformen.

Twistesystemet, rapporteringssystemet og bødesystemet sikrer tilsammen at der er en fair og struktureret proces hvis noget går galt, og at der er konsekvenser for dårlig adfærd. De automatiske baggrundstjenester markerer forsinkede lån, trækker point og sender notifikationer uden at admin skal gøre noget manuelt, hvilket gør systemet robust og skalerbart.

Hvad vi kunne have gjort anderledes

Set i bakspejlet er der et par ting vi ville have gjort anderledes hvis vi startede forfra. Vi undervurderede i starten hvor lang tid det ville tage at merge kode på tværs af gruppemedlemmerne, særligt i de perioder hvor vi arbejdede parallelt på de samme dele af systemet. Det betød at vi brugte mere tid på at løse konflikter i GitHub end forventet.

Vi ville også have planlagt databasestrukturen mere grundigt fra starten. Vi endte med at lave flere migreringer undervejs da modellerne skulle justeres efterhånden som nye features kom til, og det tog ekstra tid. Derudover var kravspecifikationen i starten ikke ambitiøs nok i forhold til hvad vi endte med at bygge- elementer som verifikationssystemet, rapporteringssystemet, support chat og de automatiske baggrundstjenester dukkede op naturligt undervejs som logiske dele af en udlejningsplatform, og det pressede tidsplanen på enkelte punkter.

Opsummering

Vi startede dette projekt med en simpel ide om at gøre det nemmere og tryggere for private at leje ting af hinanden. Det vi endte med at bygge er et system der går langt ud over den originale kravspec, med realtidschat, notifikationer, tvistehåndtering, rapporteringssystem, brugerscore, verifikation, anmeldelsessystem, brugerblokeringssystem, support chat, appeals, bødesystem og automatiske baggrundstjenester. Det er vi som gruppe meget stolte af.

Der er selvfølgelig ting der kunne være gjort bedre. Betalingsintegration med en rigtig betalingsgateway er ikke implementeret, og der er funktioner der med mere tid ville kunne poleres yderligere. Men som MVP er vi meget tilfredse med hvad vi har bygget, og vi mener at RentIt demonstrerer en solid teknisk forståelse af hvordan man designer og bygger en moderne webapplikation med Angular, ASP.NET Core, SignalR, Cloudinary og SQL Server.

Vi har lært enormt meget gennem dette projekt, ikke kun teknisk, men også om hvad det kræver at samarbejde som et team, planlægge et større projekt og tage ansvar for sin del af arbejdet. Det er erfaringer vi tager med os videre.

Bilag 10 - Kildekode

Kildekoden for både frontend og backend er tilgængelig via GitHub. Frontend er bygget i Angular og backend er bygget i ASP.NET Core Web API. Linkene nedenfor fører direkte til de respektive repositories.

Backend: <https://github.com/Chanaquid/Svendeproeven/tree/main/backend>

Backend-repositoriet indeholder ASP.NET Core Web API projektet med controllers, services, repositories, DTO'er og Entity Framework migrations.

Frontend: <https://github.com/Chanaquid/Svendeproeven/tree/main/frontend>

Frontend repository indeholder Angular-projektet med komponenter, services og al user interface logik.

Forkortelse og begreber

Nedenstående tabel forklarer de forkortelser og tekniske begreber der bruges i rapporten.

Forkortelse/ Begreber	Betydning
API	Application Programming Interface- regler for hvordan to systemer kommunikerer. Bruges mellem frontend og backend i RentIt.
CDN	Content Delivery Network- netværk af servere der leverer filer hurtigt baseret på geografisk placering. Bruges af Cloudinary.
DTO	Data Transfer Object- objekt der flytter data mellem lag i systemet uden at eksponere interne databasemodeller direkte.
HTTP/ HTTPS	Hypertext Transfer Protocol / Secure- protokol til kommunikation på internettet. HTTPS er den krypterede version.
JWT	JSON Web Token- token der udstedes ved login og beviser over for API'et at brugeren er logget ind med de rette rettigheder.
MVP	Minimum Viable Product - en version af produktet der indeholder de vigtigste funktioner, så det kan demonstreres og testes uden at alt er bygget.
ORM	Object-Relational Mapper- oversætter mellem databasetabeller og C# objekter. RentIt bruger Entity Framework Core.
REST	Representational State Transfer - en arkitekturstil for API'er der bruger standard HTTP-metoder som GET, POST, PUT og DELETE.
SignalR	Et bibliotek fra Microsoft der gør det muligt at sende data fra server til klient i realtid uden at klienten behøver at spørge først.
SPA	Single Page Application - en webapplikation der loader en gang og derefter opdaterer indholdet dynamisk uden at genindlæse siden.
SQL	Structured Query Language - det sprog der bruges til at arbejde med relationelle databaser.
UI	User Interface - brugergrænsefladen, altså det brugeren ser og interagerer med i browseren.
ASP.NET Core	Et open source web framework fra Microsoft der bruges til at bygge backend API'et i RentIt
Angular	Et komponentbaseret frontend framework fra Google der bruges til at bygge brugergrænsefladen i RentIt som en Single Page Application.

GitHub	En platform til versionsstyring der gør det muligt for flere udviklere at arbejde på den samme kodebase samtidigt uden at overskrive hinandens arbejde.
Branch	En separat kopi af koden i GitHub hvor man kan arbejde på nye features uden at påvirke hovedkoden.
Merge	Processen hvor kode fra en branch sammenføjes med hovedkoden i GitHub efter en feature er færdigudviklet.
Docker	Et værktøj der pakker applikationen i isolerede containers så den kører ens på alle maskiner uanset styresystem.
Entity Framework Core (EF Core)	En ORM fra Microsoft der oversætter mellem C# objekter og databasetabeller så man ikke behøver at skrive rå SQL.
Endpoint	En specifik URL i API'et som frontend kalder for at hente eller sende data til backend.
Hub	En SignalR klasse på serveren der håndterer realtidsforbindelser og sender beskeder til tilsluttede klienter.
Interceptor	En Angular komponent der automatisk behandler alle udgående HTTP kald, bruges i RentIt til at tilføje JWT token til alle API kald.
Kanban board	En projektstyringsmetode hvor opgaver flyttes gennem kolonner som "To Do", "Doing" og "Done" for at give overblik over arbejdets fremdrift.
Refresh Token	Et langtidsholdbart token der bruges til automatisk at udstede et nyt JWT token når det gamle udløber så brugeren ikke bliver logget ud midt i en session.
Repository Pattern	En arkitekturmetode der adskiller databasekommunikation fra forretningslogikken ved at al databaseadgang går gennem et dedikeret repository lag.
SOLID	Et sæt principper for objektorienteret softwaredesign der sikrer at koden er nem at vedligeholde, udvide og teste.

Bilag oversigt

Nedenstående tabel giver et overblik over alle bilag tilknyttet RentIt-projektet. Bilagene er enten inkluderet direkte i rapporten, vedlagt som separate dokumenter eller placeret i procesrapporten.

Bemærk: Use case diagrammet findes i to versioner - det originale fra kravspecifikationen og en opdateret version. Diagrammet er opdateret fordi systemet undervejs udviklede sig betydeligt ud over den oprindelige kravspecifikation, og den opdaterede version afspejler de funktioner der faktisk er implementeret. Begge versioner er inkluderet i Bilag 1.

Bilag	Titel	Indhold	Placering
Bilag 1	Use case Diagram (Original og opdateret)	Det originale use case diagram fra kravspecifikationen samt den opdaterede version. Diagrammet er opdateret fordi systemet udviklede sig betydeligt ud over den oprindelige kravspecifikation. Begge versioner er inkluderet for at vise udviklingen.	<i>Separat dokument</i>
Bilag 2	Use case beskrivelser	Detaljerede beskrivelser af alle use cases inkl. aktør, forudsætninger, forventet forløb og alternative forløb.	<i>Separat dokument (samme som Bilag 1)</i>
Bilag 3	Aktør kontekst diagram	Diagram der viser systemets tre aktører - Bruger, Administrator og Developers og deres relation til webserveren og databasen.	<i>Produktrapport</i>
Bilag 4	Aktør beskrivelse	Beskrivelse af alle tre aktører med navn, type, beskrivelse og forventet teknisk niveau.	<i>Produktrapport</i>
Bilag 5	GANTT diagram	Estimeret tidsplan for projektførelsen fra planlægning til aflevering, inkl. faser som implementering, UI, rapport og testing.	<i>Processrapport</i>
Bilag 6	ER Diagram	Entity-Relationship diagram over databasestrukturen med alle tabeller, relationer og nøgler.	<i>Separat dokument</i>
Bilag 7	Testrapport	Komplet testrapport med resultater fra 464 unit tests (controllers, services og repositories), manuelle Swagger API tests samt beskrivelse af brugertest og UI-test.	<i>Separat dokument</i>
Bilag 8	Brugervejledning	Vejledning til installation og anvendelse af RentIt, inkl. opstart med Docker Compose, opstart af frontend og gennemgang af de centrale brugerflows.	<i>Separat dokument</i>
Bilag 9	Logbog	Projektdagbog/logbog fra hver gruppemedlem gennem hele forløbet, samlet i et dokument	<i>Separat dokument</i>
Bilag 10	Kildekoden	Den fulde kildekode for både frontend og backend.	<i>Processrapport</i>